

Projekt nr 3 - PUS

Bartłomiej Żądło, nr albumu: 151505

Etap 1: Specyfikacja protokołu RCMP (Real-Time Chat Messaging Protocol)

1. Cel protokołu i zakres

RCMP (Real-Time Chat Messaging Protocol) to protokół warstwy aplikacyjnej przeznaczony do obsługi komunikatora czasu rzeczywistego — systemu umożliwiającego wymianę wiadomości tekstowych między użytkownikami w ramach pokoi tematycznych (ang. *rooms*) oraz w trybie wiadomości prywatnych (1:1).

Jakie problemy rozwiązuje?

- Synchroniczną wymianę wiadomości tekstowych z niskim opóźnieniem
- Zarządzanie sesjami wielu równoczesnych użytkowników
- Śledzenie statusu obecności użytkowników (online / offline / away)
- Potwierdzanie dostarczenia wiadomości (delivery acknowledgement)
- Utrzymanie połączenia przy braku aktywności (keep-alive)
- Dołączanie i opuszczanie pokoi w trakcie trwania sesji

Model działania: klient–serwer (client–server). Serwer centralny pośredniczy we wszystkich wiadomościach. Klienci nie komunikują się bezpośrednio między sobą.

2. Założenia Techniczne

Transport: TCP. Gwarantuje dostarczenie danych w kolejności, co jest wymagane dla komunikatora — nie można dopuścić do zmiany kolejności wiadomości. TLS 1.3 nakładany ponad TCP zapewnia szyfrowanie.

Kodowanie wiadomości: JSON z separatorem końca ramki \n (newline-delimited JSON). Każda wiadomość to pojedyncza linia JSON zakończona znakiem \n. Wybór JSON uzasadniony jest czytelnością podczas debugowania i łatwością walidacji schematu.

Niezawodność:

- Warstwa transportowa (TCP): gwarantuje kolejność, retransmisję na poziomie segmentów, wykrywanie utraty pakietów
- Warstwa aplikacyjna (RCMP): odpowiada za potwierdzenia dostarczenia (MESSAGE_ACK), nadawanie sekwencyjnych identyfikatorów wiadomości (seq_id), wykrywanie duplikatów, timeouty sesji i mechanizm keep-alive (PING/PONG)

3. Struktura Komunikatów

Typy wiadomości

TYP	KIERUNEK	OPIS
LOGIN	CLIENT -> SERWER	Uwierzytelnienie użytkownika
LOGIN_OK	SERWER -> CLIENT	Potwierdzenie zalogowania, wydanie tokenu sesji
LOGIN_ERR	SERWER -> CLIENT	Błąd logowania
JOIN_ROOM	CLIENT -> SERWER	Dołączenie do pokoju
LEAVE_ROOM	CLIENT -> SERWER	Opuszczenie pokoju
ROOM_EVENT	SERWER -> CLIENT	Powiadomienie o zmianie w pokoju (ktoś dołączył/wyszedł)
SEND_MESSAGE	CLIENT -> SERWER	Wysłanie wiadomości do pokoju lub użytkownika
DELIVER_MESSAGE	SERWER -> CLIENT	Dostarczenie wiadomości do odbiorcy
MESSAGE_ACK	CLIENT -> SERWER	Potwierdzenie odebrania wiadomości
STATUS	CLIENT -> SERWER	Zmiana statusu (online/away)
PING	CLIENT -> SERWER/SERWER -> CLIENT	Keep-alive
PONG	SERWER -> CLIENT/CLIENT -> SERWER	Odpowiedź na PING
ERROR	SERWER -> CLIENT	Błąd protokołu lub aplikacyjny
BYE	CLIENT -> SERWER	Zamknięcie sesji przez klienta
BYE_ACK	SERWER -> CLIENT	Potwierdzenie zamknięcia sesji

Pola wiadomości

Każda wiadomość zawiera wspólną kopertę (envelope):

```
{  
  "type": "<TYP_WIADOMOŚCI>", // wymagane  
  "msg_id": "<UUID v4>", // wymagane – unikalny ID wiadomości  
  "ts": 1715000000123, // wymagane – Unix timestamp ms  
  "token": "<session_token>", // wymagane dla wszystkich poza LOGIN  
  "payload": { ... } // opcjonalne – dane właściwe  
}
```

Przykładowe ramki

LOGIN (C → S)

```
{  
  "type": "LOGIN",  
  "msg_id": "a1b2c3d4-0001-0001-0001-000000000001",  
  "ts": 1715000000000,  
  "token": null,  
  "payload":  
  {  
    "username": "alice",  
    "password": "tajnehaslo123",  
    "nonce": "xK92mP1q"  
  }  
}
```

ERROR (S → C)

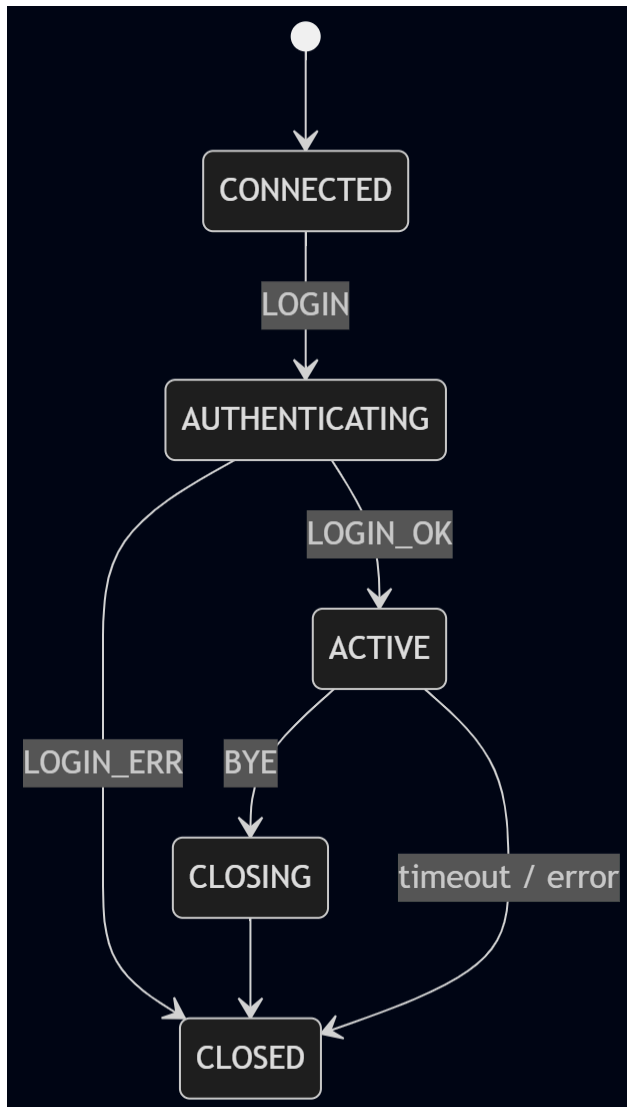
```
{  
  "type": "ERROR",  
  "msg_id": "b8c9d0e1-...",  
  "ts": 1715000300000,  
  "token": null,  
  "payload":  
  {  
    "code": 4031,  
    "message": "Invalid session token",  
    "ref_msg_id": "c3d4e5f6-..."  
  }  
}
```

Walidacja – co jest błędem formatu:

- Brak pola type, msg_id lub ts → błąd 4001 MALFORMED_ENVELOPE
- type o nieznanej wartości → błąd 4002 UNKNOWN_TYPE
- ts odbiega od czasu serwera o więcej niż ± 300 s → błąd 4003 TIMESTAMP_SKEW
- Rozmiar wiadomości przekracza 64 KB → błąd 4004 MESSAGE_TOO_LARGE
- JSON niepoprawny składniowo → rozłączenie bez odpowiedzi (ochrona przed flooding)
- Brak pola token lub token nieważny dla typów wymagających autoryzacji → błąd 4031 UNAUTHORIZED

4. Model stanów/przebieg komunikacji

Stany sesji po stronie serwera



Stany sesji po stronie klienta

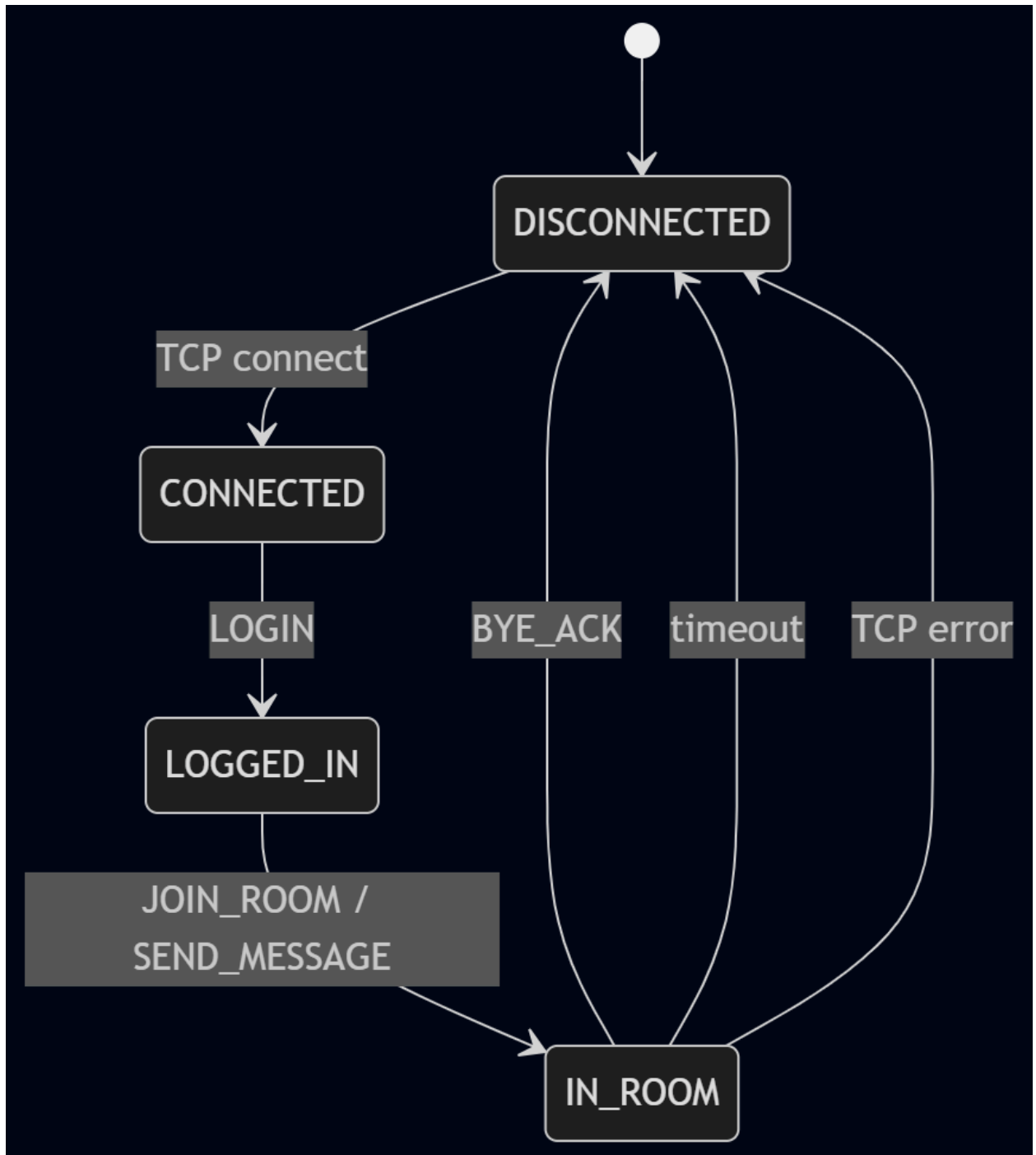
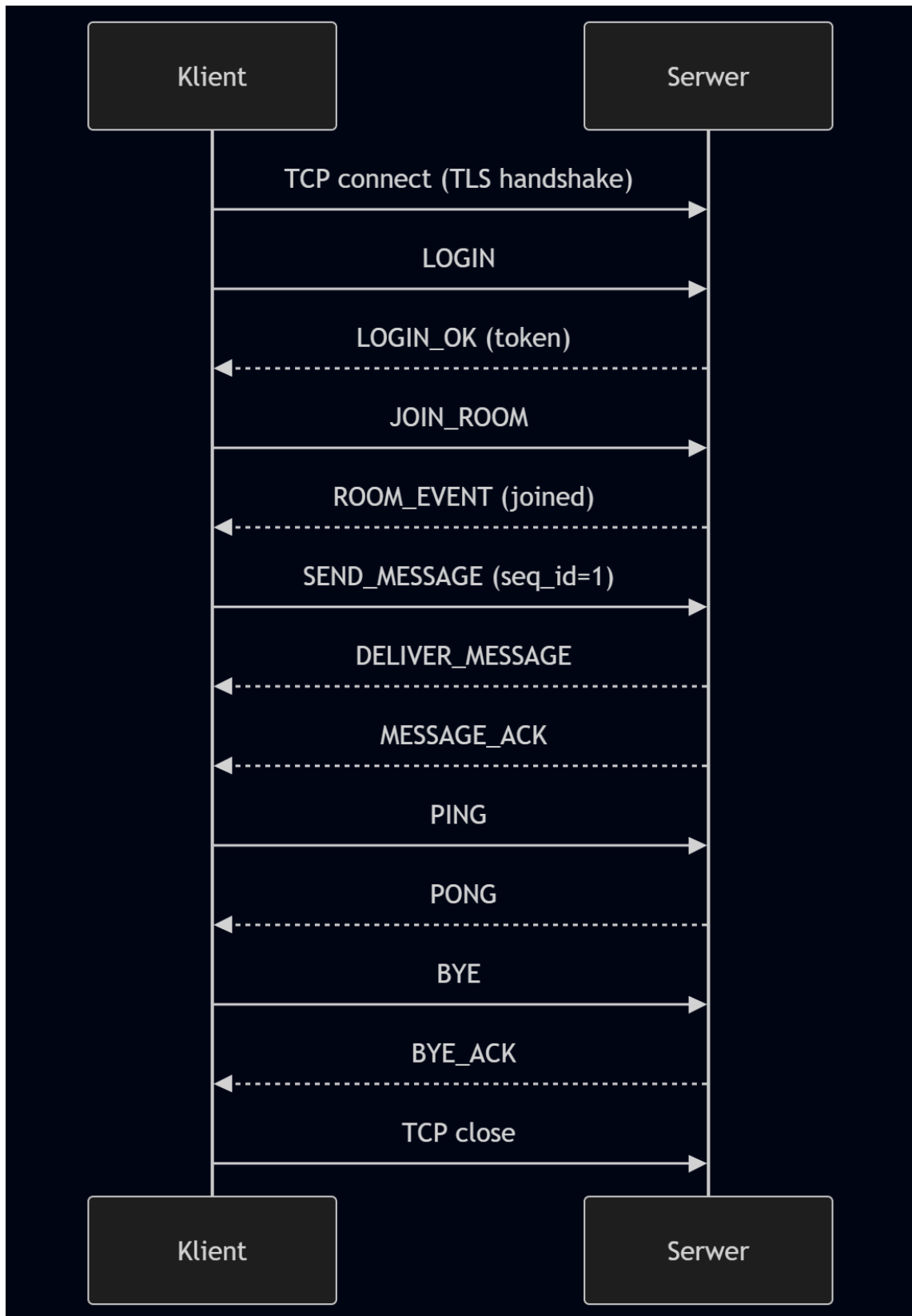


Diagram sekwencji - pełna sesja



Timeouty i keep-alive

PARAMETR	WARTOŚĆ
Timeout logowania (od połączenia TCP do LOGIN_OK)	10 s
Interwał PING (klient → serwer)	30 s
Timeout PONG (brak odpowiedzi)	10 s
Timeout sesji (brak aktywności / PING)	90 s
Timeout oczekiwania na MESSAGE_ACK	5 s
Liczba prób retransmisji SEND_MESSAGE	3

Retransmisja: jeśli klient nie otrzyma MESSAGE_ACK w ciągu 5 s, ponawia SEND_MESSAGE z tym samym msg_id i seq_id (max 3 razy). Serwer wykrywa duplikat po msg_id i odsyła MESSAGE_ACK bez ponownego dostarczenia.

5. Bezpieczeństwo

Poufność

Całe połączenie jest szyfrowane przez TLS 1.3. Dozwolone szyfry: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256. Certyfikat serwera weryfikowany przez klienta (PKI lub przypięcie certyfikatu w kliencie — certificate pinning).

Integralność wiadomości

Każda wiadomość SEND_MESSAGE zawiera pole hmac — HMAC-SHA256 wyliczony z konkatenacji msg_id + ts + seq_id + body, kluczem jest sekret wymieniany przy logowaniu (dostarczany w LOGIN_OK jako hmac_secret, szyfrowany przez TLS). Serwer weryfikuje HMAC przed przekazaniem wiadomości.

Uwierzytelnienie

- Klient wysyła username + password (plaintext) + jednorazowy nonce. Połączenie jest chronione przez TLS 1.3, więc hasło nigdy nie jest eksponowane w sieci. Serwer weryfikuje hasło względem przechowywanego bcrypt(password).
- Serwer przechowuje bcrypt(password) — weryfikuje po odebraniu hash
- Po pozytywnej weryfikacji serwer wydaje token sesji JWT (HS256, TTL 3600 s) podpisany kluczem serwera
- Token dołączany do każdej kolejnej wiadomości w polu token

Autoryzacja

- Dostęp do pokoi prywatnych wymaga zaproszenia — serwer sprawdza listę ACL pokoju przed JOIN_ROOM
- Użytkownik może wysyłać wiadomości tylko do pokoi, do których dołączył

Ochrona przed replay

- Pole nonce w LOGIN — jednorazowe, serwer przechowuje użyte nonce przez 300 s
- Pole ts — serwer odrzuca wiadomości, których timestamp odbiega od czasu serwera o więcej niż ± 300 s (błąd 4003)
- Pole msg_id (UUID v4) — serwer przechowuje odebrane msg_id przez okno 5 minut; duplikat → odrzucenie z MESSAGE_ACK bez ponownego przetwarzania

Model zagrożeń (podstawowy)

ZAGROŻENIE	MECHANIZM OCHRONY
Podśluch komunikacji	TLS 1.3
Podszywanie się pod użytkownika	JWT + HMAC, hasło chronione przez TLS (nigdy nie w plaintext w sieci)
Replay atak	nonce + timestamp + msg_id cache
Falszowanie treści wiadomości	HMAC-SHA256 na body
Brute-force logowania	Rate limit: max 5 prób LOGIN / 60 s / IP
Przepelnienie serwera (flooding)	Maks. rozmiar wiadomości 64 KB, rate limit SEND_MESSAGE
Przejęcie sesji	Token JWT z TTL, możliwość unieważnienia po BYE

6. Obsługa błędów i awarii połączenia

KOD	NAZWA	ZNACZENIE
4001	MALFORMED_ENVELOPE	Brak wymaganego pola w kopercie
4002	UNKNOWN_TYPE	Nieznany typ wiadomości
4003	TIMESTAMP_SKEW	Timestamp poza oknem ± 300 s
4004	MESSAGE_TOO_LARGE	Wiadomość przekracza 64 KB
4005	INVALID_HMAC	Błąd weryfikacji HMAC
4011	LOGIN_FAILED	Błędne dane logowania
4012	LOGIN_RATE_LIMIT	Przekroczono limit prób logowania
4031	UNAUTHORIZED	Brak lub nieważny token sesji
4032	FORBIDDEN	Brak uprawnień do zasobu (pokój prywatny)
4041	ROOM_NOT_FOUND	Pokój nie istnieje
4042	USER_NOT_FOUND	Nieznany użytkownik (wiadomość prywatna)
4291	SEND_RATE_LIMIT	Przekroczono limit wiadomości
5001	SERVER_ERROR	Wewnętrzny błąd serwera
5002	SERVER_OVERLOAD	Serwer przeciążony, spróbuj ponownie

Zachowanie po błędach

- Błąd formatu (4001–4004): serwer odsyła ERROR z odpowiednim kodem i zamyka połączenie TCP po 3 kolejnych błędach formatu w ciągu 60 s (ochrona przed złośliwym klientem)
- Błąd HMAC (4005): wiadomość odrzucona, ERROR do nadawcy, sesja utrzymana (możliwy błąd implementacji klienta)
- Błąd autoryzacji (4031): wiadomość odrzucona, klient powinien ponownie zalogować się; po 3 wystąpieniach w ciągu 60 s — rozłączenie
- Rate limit (4291): serwer ignoruje wiadomość i odsyła ERROR 4291; okno: max 30 wiadomości / 10 s / użytkownik

Timeout połączenia

- Serwer odlicza 90 s od ostatniej aktywności. Jeśli klient nie wyśle PING ani żadnej wiadomości — serwer wysyła ERROR 4031 SESSION_EXPIRED i zamyka TCP
- Klient, nie otrzymując PONG w ciągu 10 s od PING, uznaje połączenie za zerwane i inicjuje reconnect

Utrata połączenia w trakcie sesji

- Klient wykrywa zerwanie TCP (wyjątek I/O lub timeout PONG)
- Klient próbuje ponownie połączyć z wykładniczym backoff: 1 s, 2 s, 4 s, 8 s, max 60 s
- Po ponownym połączeniu klient wykonuje nowy LOGIN — serwer może opcjonalnie odtworzyć stan sesji (pokoje) na podstawie user_id z tokenu, jeśli token jest jeszcze ważny

Duplikaty i niekompletne wiadomości

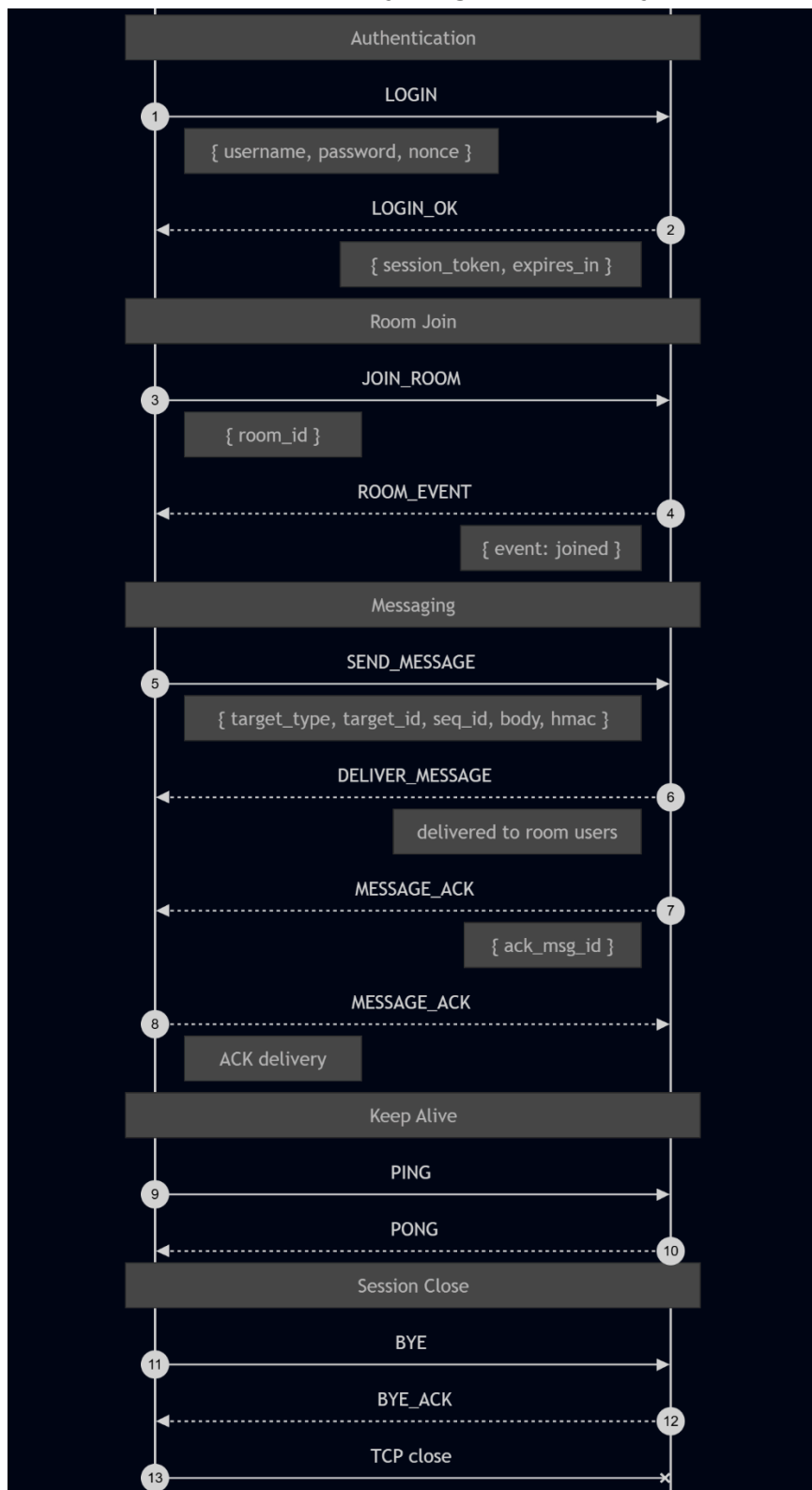
- Duplikaty wykrywane po msg_id — serwer odpowiada MESSAGE_ACK bez ponownego dostarczenia
- Niekompletna ramka JSON (przerwane połączenie w połowie wiadomości): serwer buforuje dane do wystąpienia \n; jeśli bufor przekroczy 64 KB bez \n — rozłączenie z błędem 4004
- Serwer nie przetwarza częściowych wiadomości

Limity i ochrona przed nadużyciami

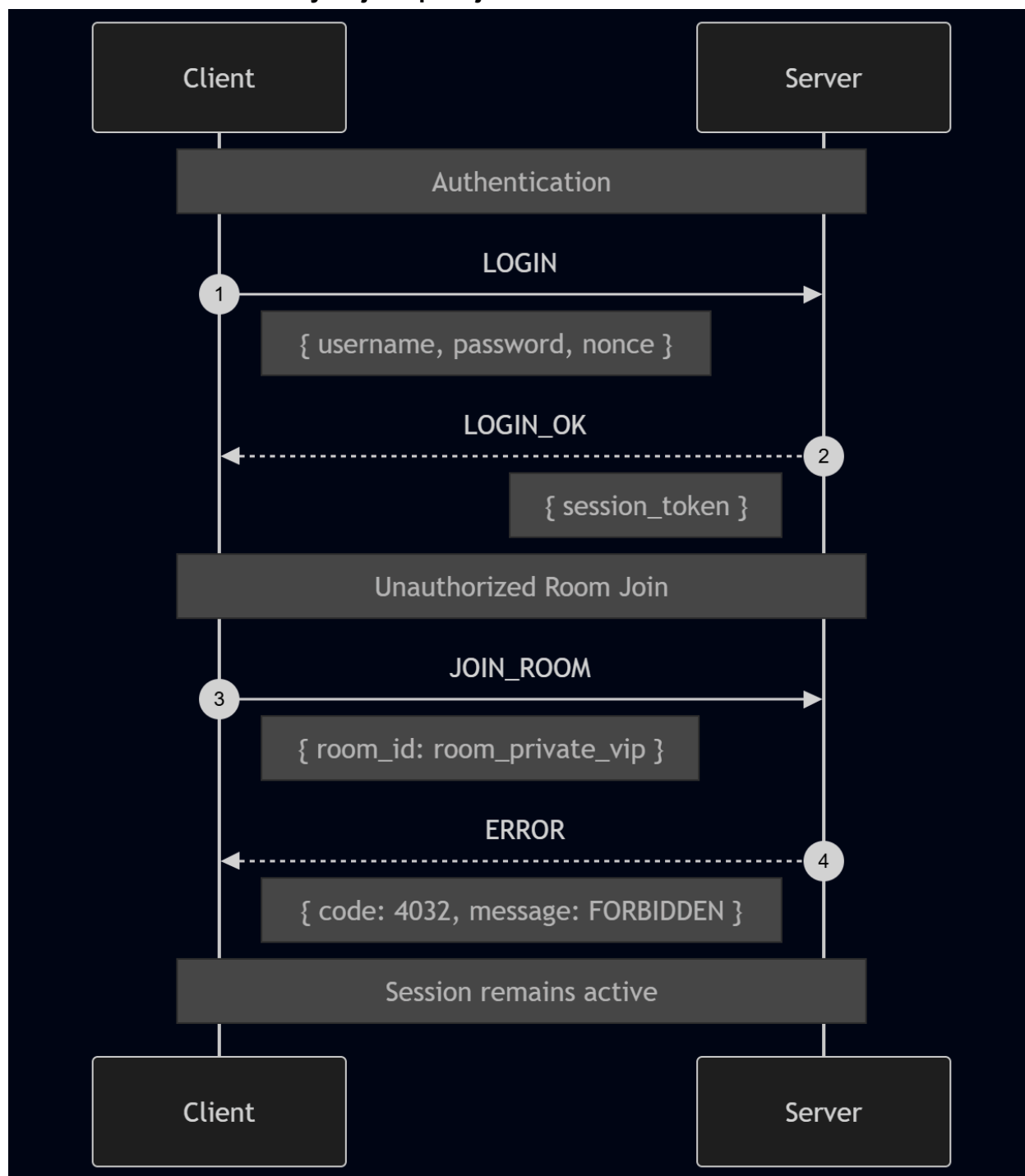
LIMIT	WARTOŚĆ
Maks. rozmiar wiadomości	64 KB
Maks. wiadomości / 10 s / użytkownik	30
Maks. prób LOGIN / 60 s / IP	5
Maks. otwartych połączeń / IP	10
Maks. pokoiów, do których należy użytkownik	50
Maks. długość nazwy użytkownika / pokoju	64 znaki

7. Przykładowe scenariusze komunikacji

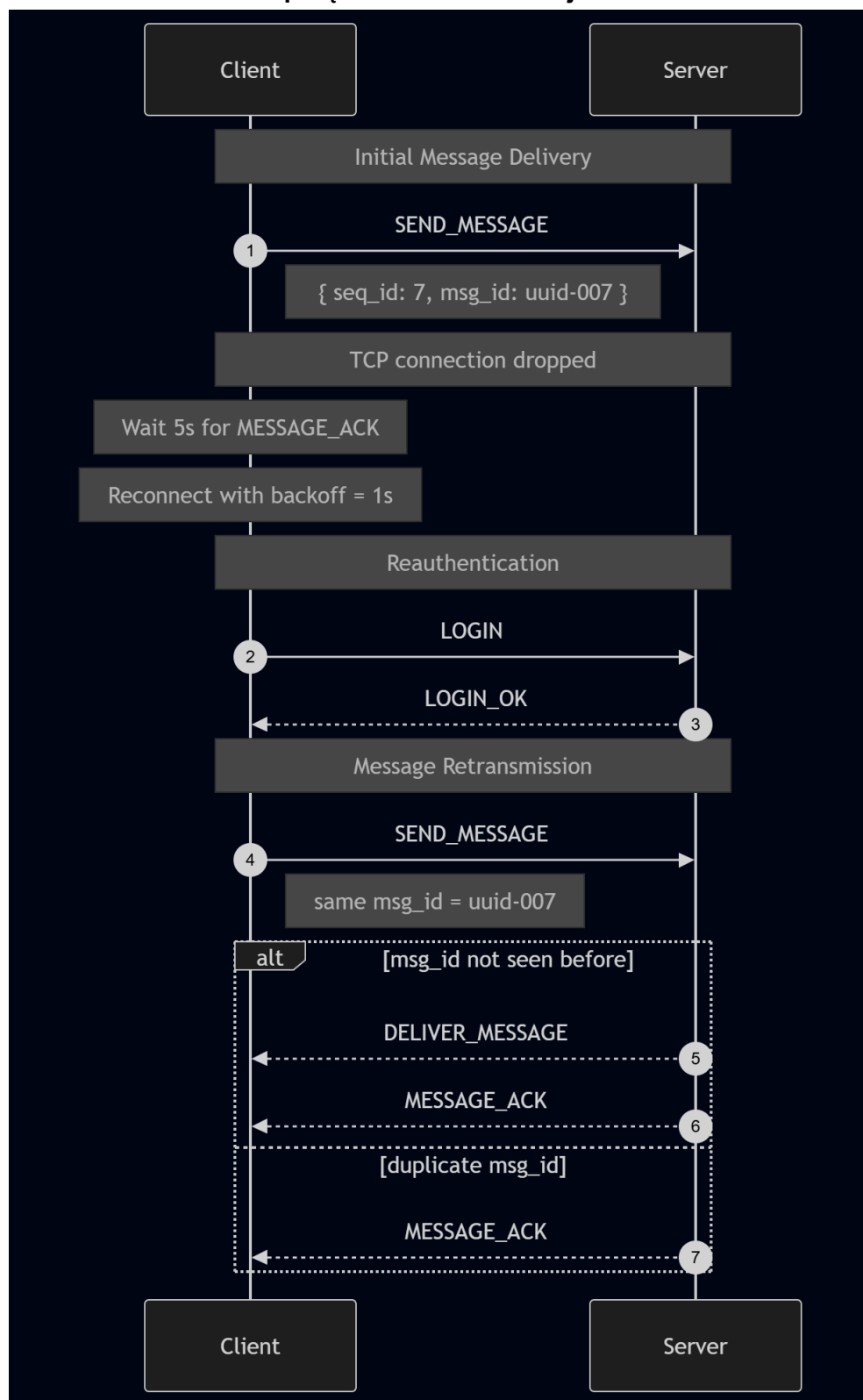
Scenariusz 1 – Poprawna sesja: logowanie, pokój, wiadomość, wylogowanie



Scenariusz 2 — Brak autoryzacji do pokoju



Scenariusz 3 — Utrata połączenia i retransmisja



Scenariusz 4 — Niepoprawny JSON / malformed envelope

